

ABE 715: Computer Programming

Compendium Part 6

Appendices, Reference Materials, and Project Guide

Prepared for:
MSc. Irrigation Engineering and Management
MSc. Agricultural and Biosystems Engineering

Instructor: Dr. Kadeghe G. Fue
Electronics and Precision Agriculture Lab

Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)

Academic Year 2025/2026

Contents

Preface ii

1 Appendix A: Python Cheat Sheet 1
- 1.1 Basic Syntax . . . 1
  - 1.1.1 Variables and Types . . . 1
  - 1.1.2 Lists (Ordered, Mutable) . . . 1
  - 1.1.3 Dictionaries (Key-Value) . . . 1
- 1.2 Control Structures . . . 1
  - 1.2.1 If-Elif-Else . . . 1
  - 1.2.2 Loops . . . 2
- 1.3 Essential Libraries . . . 2
  - 1.3.1 NumPy (Math) . . . 2
  - 1.3.2 Pandas (Data) . . . 2
  - 1.3.3 Matplotlib (Plotting) . . . 2

2 Appendix B: Engineering Algorithms 3
- 2.1 Unit Conversions . . . 3
- 2.2 Hydrology: Penman-Monteith (Simplified) . . . 3
- 2.3 Irrigation: Pump Power . . . 4
- 2.4 Soil Physics: Bulk Density . . . 5

3 Appendix C: Debugging Guide 6
- 3.1 Common Error Messages . . . 6
- 3.2 The "Print" Debugging Technique . . . 6

4 Appendix D: Final Project Guidelines 8
- 4.1 Project Overview . . . 8
- 4.2 Approved Topics . . . 8
- 4.3 Submission Requirements . . . 8
- 4.4 Rubric (Total 100 Marks) . . . 9

Preface to Part 6

This final volume of the ABE 715 Compendium series acts as a desktop reference for students completing their coursework and final projects. It aggregates key syntax, common algorithms, debugging strategies, and standard engineering formulas implemented in Python.

It is designed to be printed and kept next to your workstation as you code.

Contents of Part 6:

- **Appendix A:** Python Syntax Cheat Sheet.
- **Appendix B:** Standard Engineering Algorithms (The "Snippets" Library).
- **Appendix C:** Debugging Common Errors.
- **Appendix D:** Final Project Rubric and Submission Guidelines.

Chapter 1

Appendix A: Python Cheat Sheet

1.1 Basic Syntax

1.1.1 Variables and Types

```
x = 10              # int
y = 3.14            # float
name = "Sensor 1"   # str
is_active = True    # bool
```

1.1.2 Lists (Ordered, Mutable)

```
data = [10, 20, 30]
data.append(40)     # Add to end
x = data[0]         # Access first item (10)
y = data[-1]        # Access last item (40)
length = len(data)  # Count items
```

1.1.3 Dictionaries (Key-Value)

```
pump = {"id": 1, "flow": 500}
print(pump["flow"]) # Access value
pump["status"] = "ON" # Add new key
```

1.2 Control Structures

1.2.1 If-Elif-Else

```
if temp > 30:
    print("Hot")
elif temp > 20:
    print("Warm")
else:
```

```
    print("Cold")
```

1.2.2 Loops

```
# For Loop (Definite iteration)
for i in range(5):
    print(i) # 0, 1, 2, 3, 4

# While Loop (Indefinite iteration)
level = 0
while level < 100:
    level += 10
```

1.3 Essential Libraries

1.3.1 NumPy (Math)

```
import numpy as np
arr = np.array([1, 2, 3])
mean = np.mean(arr)
max_val = np.max(arr)
```

1.3.2 Pandas (Data)

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()                # First 5 rows
df["new_col"] = df["col"] * 2
filtered = df[df["col"] > 10]
```

1.3.3 Matplotlib (Plotting)

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.title("Title")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Chapter 2

Appendix B: Engineering Algorithms

This section contains "Copy-Paste" ready code snippets for common agricultural engineering tasks.

2.1 Unit Conversions

```
def convert_units(value, conversion_type):
    """
    Standard Engineering Conversions.
    """
    if conversion_type == "C_to_F":
        return (value * 9/5) + 32
    elif conversion_type == "in_to_mm":
        return value * 25.4
    elif conversion_type == "ha_to_m2":
        return value * 10000
    elif conversion_type == "psi_to_bar":
        return value * 0.0689476
    elif conversion_type == "hp_to_kw":
        return value * 0.7457
    else:
        return None
```

Listing 2.1: Unit Conversion Helper

2.2 Hydrology: Penman-Monteith (Simplified)

Calculating Reference Evapotranspiration (ETo) requires complex physics. Here is a function structure for it.

```
import math

def calc_eto(temp_mean, radiation, wind_speed, humidity):
    """
    Simplified FAO-56 Penman-Monteith.
    Inputs:
```

```
    - temp_mean: Mean daily temperature (C)
    - radiation: Net radiation (MJ/m2/day)
    - wind_speed: Wind speed at 2m height (m/s)
    - humidity: Relative Humidity (%)
    """
    # Psychrometric constant (kPa/C) approx at sea level
    gamma = 0.066

    # Slope of vapor pressure curve (delta)
    delta = (4098 * (0.6108 * math.exp((17.27 * temp_mean) /
            (temp_mean + 237.3)))) / ((temp_mean + 237.3) **
2)

    # This is a placeholder for the full FAO equation
    # Real implementation requires vapor pressure deficit
calc

    numerator = (0.408 * delta * radiation) + \
                (gamma * (900 / (temp_mean + 273)) *
wind_speed * humidity)

    denominator = delta + (gamma * (1 + 0.34 * wind_speed))

    eto = numerator / denominator
    return eto
```

Listing 2.2: ETo Calculator Template

2.3 Irrigation: Pump Power

```
def calc_pump_power(flow_lps, head_m, eff_pump=0.75,
eff_motor=0.9):
    """
    Calculates power requirements for an electric pump.
    Inputs:
    - flow_lps: Flow rate in Liters/Second
    - head_m: Total dynamic head in Meters
    - eff_pump: Pump efficiency (decimal)
    - eff_motor: Motor efficiency (decimal)

    Returns: Power in Kilowatts (kW)
    """
    # Water Power (kW) = (Q * H * 9.81) / 1000
    # Q must be in m3/s for standard physics, but we usually
input L/s
    flow_m3s = flow_lps / 1000.0

    water_power = (flow_m3s * head_m * 9810) / 1000.0 # Using
    Specific Weight

```

```
    brake_power = water_power / eff_pump
    total_power = brake_power / eff_motor

    return total_power
```

Listing 2.3: Pump Sizing Script

2.4 Soil Physics: Bulk Density

```
def soil_properties(dry_mass_g, total_volume_cm3,
   particle_density=2.65):
    """
    Calculates Bulk Density and Porosity.
    """
    bulk_density = dry_mass_g / total_volume_cm3

    # Porosity = 1 - (Bulk Density / Particle Density)
    porosity = (1 - (bulk_density / particle_density)) * 100

    return bulk_density, porosity
```

Listing 2.4: Soil Properties

Chapter 3

Appendix C: Debugging Guide

Even experienced engineers make mistakes. Debugging is the process of finding and fixing them.

3.1 Common Error Messages

SyntaxError
Meaning: You violated the grammar rules of Python.
Check: Missing colons `:` after `if/def/for`? Unclosed parentheses `()`?

IndentationError
Meaning: Your whitespace is inconsistent.
Check: Are you mixing tabs and spaces? Ensure all code inside a block is indented by exactly 4 spaces.

NameError
Meaning: You tried to use a variable that doesn't exist.
Check: Did you misspell the variable name? Is the variable inside a function (local scope) and you are trying to print it outside?

TypeError
Meaning: You tried to do math on incompatible types.
Check: Adding a string to a number (`"5" + 5`)? Use `int()` or `float()` to convert strings.

IndexError
Meaning: You tried to access a list item that doesn't exist.
Check: If a list has 5 items, the last index is 4 (not 5).

3.2 The "Print" Debugging Technique

The simplest way to fix logic errors (code runs but gives wrong answer) is to print variable values at intermediate steps.

```
def calculate_area(r):
    # Debugging: Check if radius is correct
    print(f"DEBUG: Radius received is {r}")
```

```
4
5       area = 3.14 * r * 2 # Mistake! Should be r**2
6
7       # Debugging: Check intermediate calculation
8       print(f"DEBUG: Calculated area is {area}")
9
10      return area
```

Chapter 4

Appendix D: Final Project Guidelines

4.1 Project Overview

The final project is a group assignment (max 3 students) that integrates Python programming with Agricultural Engineering principles. You must identify a specific problem and build a software solution.

4.2 Approved Topics

1. **Smart Irrigation Scheduler:** A system that reads weather CSV data, calculates ETo, checks soil moisture thresholds, and outputs a watering schedule.
2. **Yield Mapping Tool:** A script that reads GPS yield monitor data, cleans outliers using Pandas, and generates a heatmap using Seaborn/GeoPandas.
3. **Greenhouse IoT Simulation:** A program that simulates temperature/humidity changes and "actuates" fans/heaters using Python logic and classes.
4. **Hydraulic Network Solver:** A tool that solves the Hardy Cross method or Pipe Friction loss for a simple irrigation network.

4.3 Submission Requirements

You must submit a zipped folder containing:

1. **Source Code (.py or .ipynb):** Well-commented, following PEP8 style guide.
2. **Technical Report (PDF):** 5-10 pages.
    - Introduction to the problem.
    - Methodology (Algorithm Flowchart).
    - Results (Screenshots of the program running, graphs).
    - Conclusion.
3. **Presentation Slides:** 10 slides for the final defense.

4.4 Rubric (Total 100 Marks)

Component	Marks	Criteria
Functionality	40	Does the code run without errors? Does it solve the problem accurately?
Code Quality	20	Variable naming, comments, modularity (use of functions), efficiency.
Complexity	20	Use of loops, lists, libraries (Pandas/NumPy), and logic depth.
Report	20	Clarity of writing, quality of flowcharts and diagrams.

Good Luck!
Electronics and Precision Agriculture Lab
Sokoine University of Agriculture